

app.py

```
from flask import Flask, request, jsonify, send_from_directory
from flask_cors import CORS
import tensorflow as tf
import numpy as np
import cv2
import os

app = Flask(__name__, static_folder='static')
CORS(app)

IMG_SIZE = (128, 128)
CLASS_NAMES = ['angry', 'happy', 'relaxed', 'sad']

mobilenet_model = tf.keras.models.load_model('model/best_model.keras')
cnn_model = tf.keras.models.load_model('model/cnn_model.keras')
print("✅ Both models loaded.")

def preprocess(image_bytes):
    np_arr = np.frombuffer(image_bytes, np.uint8)
    img_bgr = cv2.imdecode(np_arr, cv2.IMREAD_COLOR)
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    img_resized = cv2.resize(img_rgb, IMG_SIZE)
    img_array = img_resized.astype('float32') / 255.0
    return np.expand_dims(img_array, axis=0)

def run_predict(model, img_array):
    predictions = model.predict(img_array, verbose=0)[0]
    predicted_class = CLASS_NAMES[int(np.argmax(predictions))]
    confidence = float(np.max(predictions)) * 100
    probs = {CLASS_NAMES[i]: round(float(predictions[i]) * 100, 1) for i in
range(len(CLASS_NAMES))}
    return predicted_class, round(confidence, 1), probs

@app.route('/')
def index():
    return send_from_directory('static', 'index.html')

@app.route('/predict', methods=['POST'])
def predict():
    if 'image' not in request.files:
        return jsonify({'error': 'No image uploaded'}), 400

    image_bytes = request.files['image'].read()

    try:
        img_array = preprocess(image_bytes)

        mob_emotion, mob_conf, mob_probs = run_predict(mobilenet_model, img_array)
```

```

cnn_emotion, cnn_conf, cnn_probs = run_predict(cnn_model, img_array)

return jsonify({
    'mobilenet': {
        'emotion':    mob_emotion,
        'confidence': mob_conf,
        'probabilities': mob_probs
    },
    'cnn': {
        'emotion':    cnn_emotion,
        'confidence': cnn_conf,
        'probabilities': cnn_probs
    }
})
except Exception as e:
    return jsonify({'error': str(e)}), 500

if __name__ == '__main__':
    app.run(debug=True, port=5000)

```

model_training.py

```

import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import MobileNetV2
from sklearn.utils.class_weight import compute_class_weight
import numpy as np
import os

IMG_SIZE = (128, 128)
BATCH_SIZE = 32

# — Data Generators
——
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=10,
    width_shift_range=0.1,
    height_shift_range=0.1,
    zoom_range=0.1,
    horizontal_flip=True,
    brightness_range=[0.8, 1.2]
)

val_datagen = ImageDataGenerator(rescale=1./255)
test_datagen = ImageDataGenerator(rescale=1./255)

```

```
train_data = train_datagen.flow_from_directory(
    'dataset/train',
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical'
)
```

```
val_data = val_datagen.flow_from_directory(
    'dataset/val',
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical'
)
```

```
test_data = test_datagen.flow_from_directory(
    'dataset/test',
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    shuffle=False
)
```

```
# — Class Names (IMPORTANT)
```

```
CLASS_NAMES = list(train_data.class_indices.keys())
print("Class order:", CLASS_NAMES)
```

```
# — Class Weights (handles imbalance)
```

```
class_weights = compute_class_weight(
    'balanced',
    classes=np.unique(train_data.classes),
    y=train_data.classes
)
class_weights = dict(enumerate(class_weights))
print("Class weights:", class_weights)
```

```
# — Model (Transfer Learning)
```

```
base_model = MobileNetV2(
    weights='imagenet',
    include_top=False,
    input_shape=(128, 128, 3)
)
```

```
base_model.trainable = False # freeze base first
```

```
model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.BatchNormalization(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(len(CLASS_NAMES), activation='softmax')
])
```

```
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
model.summary()
```

```
# — Callbacks
```

```
os.makedirs('model', exist_ok=True)
```

```
checkpoint = tf.keras.callbacks.ModelCheckpoint(
    'model/best_model.keras',
    monitor='val_accuracy',
    save_best_only=True,
    verbose=1
)
```

```
early_stop = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True
)
```

```
reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=3,
    verbose=1
)
```

```
# — Training Phase 1
```

```

print("\n🚀 Training Phase 1...\n")

history = model.fit(
    train_data,
    epochs=30,
    validation_data=val_data,
    callbacks=[checkpoint, early_stop, reduce_lr],
    class_weight=class_weights
)

# — Fine-Tuning Phase
—
print("\n🔧 Fine-tuning...\n")

base_model.trainable = True

# freeze most layers, only train last 20
for layer in base_model.layers[:-20]:
    layer.trainable = False

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history_fine = model.fit(
    train_data,
    epochs=30,
    validation_data=val_data,
    callbacks=[checkpoint, early_stop, reduce_lr],
    class_weight=class_weights
)

import json

# Save phase 1 history
with open('model/mobilenet_history.json', 'w') as f:
    json.dump(history.history, f)

# Save fine-tune history
with open('model/mobilenet_history_fine.json', 'w') as f:
    json.dump(history_fine.history, f)

# — Load Best Model
—

```

```
model = tf.keras.models.load_model('model/best_model.keras')
```

```
# — Final Evaluation
```

```
test_loss, test_acc = model.evaluate(test_data, verbose=0)
```

```
print("\n===== FINAL RESULTS =====")
print(f"Test Accuracy: {test_acc * 100:.1f}%")
print(f"Test Loss: {test_loss:.4f}")
print("===== \n")
```

```
# — Save Final Copy (optional backup)
```

```
model.save('model/final_best_model.keras')
print("✅ Final best model saved as 'final_best_model.keras'")
```

train_cnn.py

```
import tensorflow as tf
from tensorflow.keras import layers, models, regularizers
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from sklearn.utils.class_weight import compute_class_weight
import numpy as np
import os
```

```
IMG_SIZE = (128, 128)
BATCH_SIZE = 16
CLASS_NAMES = ['angry', 'happy', 'relaxed', 'sad']
DATASET_PATH = 'dataset'
```

```
# — Data Generators
```

```
train_datagen = ImageDataGenerator(
    rescale = 1./255,
    rotation_range = 25,
    width_shift_range = 0.2,
    height_shift_range = 0.2,
    horizontal_flip = True,
    zoom_range = 0.25,
    shear_range = 0.15,
    brightness_range = [0.7, 1.3],
    channel_shift_range = 0.1,
```

```

    fill_mode      = 'nearest'
)

val_datagen = ImageDataGenerator(rescale=1./255)
test_datagen = ImageDataGenerator(rescale=1./255)

train_data = train_datagen.flow_from_directory(
    f'{DATASET_PATH}/train',
    target_size = IMG_SIZE,
    batch_size  = BATCH_SIZE,
    class_mode  = 'categorical'
)

val_data = val_datagen.flow_from_directory(
    f'{DATASET_PATH}/val',
    target_size = IMG_SIZE,
    batch_size  = BATCH_SIZE,
    class_mode  = 'categorical'
)

test_data = test_datagen.flow_from_directory(
    f'{DATASET_PATH}/test',
    target_size = IMG_SIZE,
    batch_size  = BATCH_SIZE,
    class_mode  = 'categorical',
    shuffle     = False
)

```

— Class Weights

```

class_weights = compute_class_weight(
    'balanced',
    classes = np.unique(train_data.classes),
    y       = train_data.classes
)
class_weights = dict(enumerate(class_weights))

```

— Model Architecture

```

def conv_block(x, filters, dropout_rate):
    x = layers.Conv2D(
        filters, (3, 3),
        activation = 'relu',
        padding    = 'same',
        kernel_regularizer = regularizers.l2(1e-4)
    )(x)
    x = layers.BatchNormalization()(x)
    x = layers.Conv2D(
        filters, (3, 3),

```

```

        activation      = 'relu',
        padding         = 'same',
        kernel_regularizer = regularizers.l2(1e-4)
    )(x)
    x = layers.BatchNormalization()(x)
    x = layers.MaxPooling2D(2, 2)(x)
    x = layers.SpatialDropout2D(dropout_rate)(x)
    return x

inputs = layers.Input(shape=(128, 128, 3))

x = conv_block(inputs, 32, 0.2)
x = conv_block(x, 64, 0.25)
x = conv_block(x, 128, 0.3)
x = conv_block(x, 256, 0.3)
x = conv_block(x, 512, 0.35)

x = layers.GlobalAveragePooling2D()(x)

x = layers.Dense(
    256, activation='relu',
    kernel_regularizer = regularizers.l2(1e-4)
)(x)
x = layers.BatchNormalization()(x)
x = layers.Dropout(0.5)(x)

x = layers.Dense(
    128, activation='relu',
    kernel_regularizer = regularizers.l2(1e-4)
)(x)
x = layers.Dropout(0.4)(x)

outputs = layers.Dense(4, activation='softmax')(x)

model = models.Model(inputs, outputs)

model.compile(
    optimizer = tf.keras.optimizers.Adam(learning_rate=0.0001),
    loss      = tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.1),
    metrics   = ['accuracy']
)

model.summary()

# — Callbacks
_____

os.makedirs('model', exist_ok=True)

checkpoint = tf.keras.callbacks.ModelCheckpoint(
    'model/cnn_model.keras',

```

```

    monitor      = 'val_accuracy',
    save_best_only = True,
    verbose      = 1
)

reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(
    monitor = 'val_loss',
    factor   = 0.5,
    min_lr   = 1e-7,
    verbose  = 1
)

warmup_scheduler = tf.keras.callbacks.LearningRateScheduler(
    lambda epoch, lr: lr if epoch >= 5 else 1e-5 + (1e-4 - 1e-5) * (epoch / 5),
    verbose = 1
)

# — Training


---



history = model.fit(
    train_data,
    epochs      = 80,
    validation_data = val_data,
    callbacks    = [checkpoint, reduce_lr, warmup_scheduler],
    class_weight = class_weights
)

# — Evaluation


---



best_model = tf.keras.models.load_model('model/cnn_model.keras')
test_loss, test_acc = best_model.evaluate(test_data, verbose=0)

print(f"\n✅ Test Accuracy: {test_acc * 100:.1f}%")
print(f"✅ Test Loss:    {test_loss:.4f}")

```

comparison.py

```

import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from sklearn.metrics import (
    confusion_matrix, f1_score,
    roc_curve, auc
)

```

```

from sklearn.preprocessing import label_binarize
from matplotlib.colors import LinearSegmentedColormap
import seaborn as sns
import os

IMG_SIZE = (128, 128)
BATCH_SIZE = 32
CLASS_NAMES = ['angry', 'happy', 'relaxed', 'sad']
N_CLASSES = len(CLASS_NAMES)

MOB_COLOR = '#610c04'
CNN_COLOR = '#e3242b'
BG = '#1e1e2e'
SURFACE = '#2a2a3e'
TEXT_DIM = '#aaaaaa'
CLASS_COLORS = ['#f97316', '#22c55e', '#3b82f6', '#a855f7']

red_cmap = LinearSegmentedColormap.from_list(
    'custom_red', ['#2a2a3e', '#610c04', '#e3242b'], N=256
)

print("Loading models...")
mobilenet_model = tf.keras.models.load_model('model/best_model.keras')
cnn_model = tf.keras.models.load_model('model/cnn_model.keras')
print("Both models loaded.")

test_datagen = tf.keras.preprocessing.image.ImageDataGenerator(rescale=1./255)
test_data = test_datagen.flow_from_directory(
    'dataset/test',
    target_size = IMG_SIZE,
    batch_size = BATCH_SIZE,
    class_mode = 'categorical',
    shuffle = False
)

mob_loss, mob_acc = mobilenet_model.evaluate(test_data, verbose=1)
test_data.reset()

cnn_loss, cnn_acc = cnn_model.evaluate(test_data, verbose=1)
test_data.reset()

mob_probs = mobilenet_model.predict(test_data, verbose=0)
test_data.reset()
cnn_probs = cnn_model.predict(test_data, verbose=0)

true_labels = test_data.classes
mob_preds = np.argmax(mob_probs, axis=1)
cnn_preds = np.argmax(cnn_probs, axis=1)
true_labels_bin = label_binarize(true_labels, classes=list(range(N_CLASSES)))

mob_cm = confusion_matrix(true_labels, mob_preds)
cnn_cm = confusion_matrix(true_labels, cnn_preds)

def per_class_acc(cm):

```

```

return (cm.diagonal() / cm.sum(axis=1)) * 100

mob_class_acc = per_class_acc(mob_cm)
cnn_class_acc = per_class_acc(cnn_cm)

mob_f1_macro = f1_score(true_labels, mob_preds, average='macro')
cnn_f1_macro = f1_score(true_labels, cnn_preds, average='macro')
mob_f1_per = f1_score(true_labels, mob_preds, average=None)
cnn_f1_per = f1_score(true_labels, cnn_preds, average=None)

def compute_roc(probs, true_bin):
    fpr_all, tpr_all, roc_auc_all = {}, {}, {}
    for i in range(N_CLASSES):
        fpr_all[i], tpr_all[i], _ = roc_curve(true_bin[:, i], probs[:, i])
        roc_auc_all[i] = auc(fpr_all[i], tpr_all[i])
    all_fpr = np.unique(np.concatenate([fpr_all[i] for i in range(N_CLASSES)]))
    mean_tpr = np.zeros_like(all_fpr)
    for i in range(N_CLASSES):
        mean_tpr += np.interp(all_fpr, fpr_all[i], tpr_all[i])
    mean_tpr /= N_CLASSES
    macro_auc = auc(all_fpr, mean_tpr)
    return fpr_all, tpr_all, roc_auc_all, all_fpr, mean_tpr, macro_auc

mob_fpr, mob_tpr, mob_roc_auc, mob_mfpr, mob_mtpr, mob_macro_auc =
compute_roc(mob_probs, true_labels_bin)
cnn_fpr, cnn_tpr, cnn_roc_auc, cnn_mfpr, cnn_mtpr, cnn_macro_auc =
compute_roc(cnn_probs, true_labels_bin)

acc_winner = "MobileNetV2" if mob_acc > cnn_acc else "CNN"
loss_winner = "MobileNetV2" if mob_loss < cnn_loss else "CNN"
f1_winner = "MobileNetV2" if mob_f1_macro > cnn_f1_macro else "CNN"
auc_winner = "MobileNetV2" if mob_macro_auc > cnn_macro_auc else "CNN"
best_mob_cls = CLASS_NAMES[int(np.argmax(mob_f1_per))].capitalize()
best_cnn_cls = CLASS_NAMES[int(np.argmax(cnn_f1_per))].capitalize()
worst_mob_cls = CLASS_NAMES[int(np.argmin(mob_f1_per))].capitalize()
worst_cnn_cls = CLASS_NAMES[int(np.argmin(cnn_f1_per))].capitalize()
winners = [acc_winner, loss_winner, f1_winner, auc_winner]
overall = ("MobileNetV2 ✓" if winners.count("MobileNetV2") >= 3
           else "CNN ✓" if winners.count("CNN") >= 3
           else "Mixed — no single dominant model")

plt.style.use('dark_background')

fig = plt.figure(figsize=(24, 13), facecolor=BG)

gs = gridspec.GridSpec(
    3, 4, figure=fig,
    hspace=0.48, wspace=0.32,
    top=0.95, bottom=0.18,
    left=0.05, right=0.98
)

def style_ax(ax):
    ax.set_facecolor(SURFACE)

```

```

ax.tick_params(colors='white', labelsz=8)
for spine in ['top', 'right']:
    ax.spines[spine].set_visible(False)
for spine in ['bottom', 'left']:
    ax.spines[spine].set_color('#444')

ax1 = fig.add_subplot(gs[0, 0])
style_ax(ax1)
bars = ax1.bar(['MobileNetV2', 'CNN'], [mob_acc * 100, cnn_acc * 100],
               color=[MOB_COLOR, CNN_COLOR], width=0.5, edgecolor='none')
for bar, val in zip(bars, [mob_acc * 100, cnn_acc * 100]):
    ax1.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.5,
             f'{val:.1f}%', ha='center', va='bottom', color='white', fontsize=10, fontweight='bold')
ax1.set_ylim(0, 115)
ax1.set_ylabel('Accuracy (%)', color='white', fontsize=8)
ax1.set_title('Overall Accuracy', color='white', fontsize=10)
ax1.tick_params(axis='x', labelsz=8, colors='white')

ax2 = fig.add_subplot(gs[0, 1])
style_ax(ax2)
bars2 = ax2.bar(['MobileNetV2', 'CNN'], [mob_loss, cnn_loss],
                color=[MOB_COLOR, CNN_COLOR], width=0.5, edgecolor='none')
for bar, val in zip(bars2, [mob_loss, cnn_loss]):
    ax2.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.002,
             f'{val:.4f}', ha='center', va='bottom', color='white', fontsize=10, fontweight='bold')
ax2.set_ylabel('Loss', color='white', fontsize=8)
ax2.set_title('Overall Loss', color='white', fontsize=10)
ax2.tick_params(axis='x', labelsz=8, colors='white')

ax3 = fig.add_subplot(gs[0, 2])
style_ax(ax3)
x = np.arange(N_CLASSES)
ax3.bar(x - 0.175, mob_class_acc, 0.35, label='MobileNetV2', color=MOB_COLOR,
        edgecolor='none')
ax3.bar(x + 0.175, cnn_class_acc, 0.35, label='CNN', color=CNN_COLOR,
        edgecolor='none')
ax3.set_xticks(x)
ax3.set_xticklabels([c.capitalize() for c in CLASS_NAMES], color='white', fontsize=8)
ax3.set_ylabel('Accuracy (%)', color='white', fontsize=8)
ax3.set_title('Per-Class Accuracy', color='white', fontsize=10)
ax3.set_ylim(0, 120)
ax3.legend(facecolor=SURFACE, edgecolor='#444', labelcolor='white', fontsize=7)

ax4 = fig.add_subplot(gs[0, 3])
style_ax(ax4)
x = np.arange(N_CLASSES)
ax4.bar(x - 0.175, mob_f1_per, 0.35, label='MobileNetV2', color=MOB_COLOR,
        edgecolor='none')
ax4.bar(x + 0.175, cnn_f1_per, 0.35, label='CNN', color=CNN_COLOR,
        edgecolor='none')
for xi, (mv, cv) in enumerate(zip(mob_f1_per, cnn_f1_per)):
    ax4.text(xi - 0.175, mv + 0.01, f'{mv:.2f}', ha='center', va='bottom', color='white',
             fontsize=6.5)

```

```

    ax4.text(xi + 0.175, cv + 0.01, f'{cv:.2f}', ha='center', va='bottom', color='white',
            fontsize=6.5)
    ax4.set_xticks(x)
    ax4.set_xticklabels([c.capitalize() for c in CLASS_NAMES], color='white', fontsize=8)
    ax4.set_ylabel('F1 Score', color='white', fontsize=8)
    ax4.set_title('Per-Class F1 Score', color='white', fontsize=10)
    ax4.set_ylim(0, 1.2)
    ax4.legend(facecolor=SURFACE, edgecolor='#444', labelcolor='white', fontsize=7)

def draw_cm(ax, cm, title):
    display = np.zeros_like(cm, dtype=float)
    np.fill_diagonal(display, 1.0)

    sns.heatmap(display, annot=cm, fmt='d',
                cmap=LinearSegmentedColormap.from_list('tp', [SURFACE, '#610c04',
                '#e3242b'], N=256),
                vmin=0, vmax=1,
                xticklabels=[c.capitalize() for c in CLASS_NAMES],
                yticklabels=[c.capitalize() for c in CLASS_NAMES],
                ax=ax, cbar=False, linewidths=0.5, linecolor=BG,
                annot_kws={"size": 8, "color": "white"})

    ax.set_title(title, color='white', fontsize=10)
    ax.set_xlabel('Predicted', color=TEXT_DIM, fontsize=8)
    ax.set_ylabel('Actual', color=TEXT_DIM, fontsize=8)
    ax.tick_params(colors='white', labelsz=7)

ax5 = fig.add_subplot(gs[1, 0])
draw_cm(ax5, mob_cm, 'MobileNetV2 — Confusion Matrix')

ax6 = fig.add_subplot(gs[1, 1])
draw_cm(ax6, cnn_cm, 'CNN — Confusion Matrix')

ax7 = fig.add_subplot(gs[1, 2])
ax7.set_facecolor(SURFACE)
for i in range(N_CLASSES):
    ax7.plot(mob_fpr[i], mob_tpr[i], color=CLASS_COLORS[i], lw=1.5,
            label=f'{CLASS_NAMES[i].capitalize()} (AUC={mob_roc_auc[i]:.2f})')
ax7.plot(mob_mfpr, mob_mtp, color='white', lw=2, linestyle='--',
        label=f'Macro avg (AUC={mob_macro_auc:.2f})')
ax7.plot([0, 1], [0, 1], 'gray', lw=1, linestyle=':')
ax7.set_xlim([0, 1])
ax7.set_ylim([0, 1.05])
ax7.set_xlabel('False Positive Rate', color=TEXT_DIM, fontsize=8)
ax7.set_ylabel('True Positive Rate', color=TEXT_DIM, fontsize=8)
ax7.set_title('ROC Curve — MobileNetV2', color='white', fontsize=10)
ax7.legend(facecolor=SURFACE, edgecolor='#444', labelcolor='white', fontsize=7,
        loc='lower right')
ax7.tick_params(colors='white', labelsz=7)
for sp in ['top', 'right']: ax7.spines[sp].set_visible(False)
for sp in ['bottom', 'left']: ax7.spines[sp].set_color('#444')

ax8 = fig.add_subplot(gs[1, 3])
ax8.set_facecolor(SURFACE)

```

```

for i in range(N_CLASSES):
    ax8.plot(cnn_fpr[i], cnn_tpr[i], color=CLASS_COLORS[i], lw=1.5,
             label=f'{CLASS_NAMES[i].capitalize()} (AUC={cnn_roc_auc[i]:.2f})')
ax8.plot(cnn_mfpr, cnn_mtptr, color='white', lw=2, linestyle='--',
         label=f'Macro avg (AUC={cnn_macro_auc:.2f})')
ax8.plot([0, 1], [0, 1], 'gray', lw=1, linestyle=':')
ax8.set_xlim([0, 1])
ax8.set_ylim([0, 1.05])
ax8.set_xlabel('False Positive Rate', color=TEXT_DIM, fontsize=8)
ax8.set_ylabel('True Positive Rate', color=TEXT_DIM, fontsize=8)
ax8.set_title('ROC Curve — CNN', color='white', fontsize=10)
ax8.legend(facecolor=SURFACE, edgecolor='#444', labelcolor='white', fontsize=7,
           loc='lower right')
ax8.tick_params(colors='white', labelsz=7)
for sp in ['top', 'right']: ax8.spines[sp].set_visible(False)
for sp in ['bottom', 'left']: ax8.spines[sp].set_color('#444')

ax9 = fig.add_subplot(gs[2, 0])
style_ax(ax9)
delta = mob_class_acc - cnn_class_acc
d_colors = [MÖB_CÖLÖR if v >= 0 else CNN_CÖLÖR for v in delta]
bars6 = ax9.bar([c.capitalize() for c in CLASS_NAMES], delta, color=d_colors,
                edgecolor='none')
ax9.axhline(0, color='white', linewidth=0.8, linestyle='--', alpha=0.5)
for bar, val in zip(bars6, delta):
    off = 0.4 if val >= 0 else -0.4
    ax9.text(bar.get_x() + bar.get_width() / 2, val + off,
             f'{val:+.1f}%', ha='center',
             va='bottom' if val >= 0 else 'top',
             color='white', fontsize=8)
ax9.set_ylabel('MobileNetV2 – CNN (%)', color='white', fontsize=8)
ax9.set_title('Accuracy Delta\n(+ = MobileNetV2 wins)', color='white', fontsize=10)

ax_text = fig.add_subplot(gs[2, 1:])
ax_text.set_facecolor(SURFACE)
ax_text.axis('off')

summary = (
    f'EVALUATION SUMMARY\n\n'
    f'Accuracy : MobileNetV2 {mob_acc*100:.2f}% vs CNN {cnn_acc*100:.2f}% —
{acc_winner} leads by {abs(mob_acc - cnn_acc)*100:.2f}pp\n'
    f'Loss : MobileNetV2 {mob_loss:.4f} vs CNN {cnn_loss:.4f} — {loss_winner} has
lower loss\n'
    f'F1 Macro : MobileNetV2 {mob_f1_macro:.4f} vs CNN {cnn_f1_macro:.4f} —
{f1_winner} scores higher\n'
    f'ROC-AUC : MobileNetV2 {mob_macro_auc:.4f} vs CNN {cnn_macro_auc:.4f} —
{auc_winner} discriminates better\n\n'
    f'MobileNetV2 → strongest: {best_mob_cls} (F1={max(mob_f1_per):.2f}) weakest:
{worst_mob_cls} (F1={min(mob_f1_per):.2f})\n'
    f'CNN → strongest: {best_cnn_cls} (F1={max(cnn_f1_per):.2f}) weakest:
{worst_cnn_cls} (F1={min(cnn_f1_per):.2f})\n\n'
    f'Overall winner (Accuracy + Loss + F1 + AUC) : {overall}'
)

```

```
ax_text.text(0.03, 0.95, summary, transform=ax_text.transAxes,  
             color='white', fontsize=9, va='top', ha='left',  
             linespacing=1.75, family='monospace',  
             bbox=dict(facecolor='#1e1e2e', edgecolor='#555', boxstyle='round,pad=0.6'))  
  
os.makedirs('results', exist_ok=True)  
plt.savefig('results/model_comparison.png', dpi=150, bbox_inches='tight', facecolor=BG)  
print("Saved -> results/model_comparison.png")  
plt.show()
```

requirements.txt

```
flask  
flask-cors  
tensorflow  
opencv-python  
numpy
```